

Land-Use Classification from Satellite Imagery Using CNNs

Custom CNN versus Transfer Learning, with Robustness to Real-World Image Degradations

Jake Garrison | ECEN 5743 — Deep Learning | Final Project Report | May 5, 2026

1. Introduction

This project investigates deep learning for land-use classification from satellite imagery and evaluates how well the resulting models hold up under real-world image degradations. The work is motivated by autonomous aerial navigation in GPS-denied environments: a drone that cannot trust its GPS still needs to recognize the terrain below, and such a system is only useful if it survives noise, blur, low light, and arbitrary flight headings. Robustness, not just clean-data accuracy, is therefore a first-class concern.

Three convolutional architectures were trained on the EuroSAT dataset of 27,000 Sentinel-2 images: a small custom CNN trained from scratch, a frozen ImageNet-pretrained ResNet18, and a fully fine-tuned ResNet18. All three were evaluated on clean test data, stress-tested across five categories of image degradation, and benchmarked for inference latency on CPU and GPU. The remainder of this report covers project schedule (§2), dataset (§3), architectures and training (§4), experimental setup (§5), results (§6), conclusions (§7), and references (§8). Appendix A presents the most distinctive code listings.

2. Project Schedule

Figure 1 summarizes the project schedule, tracked through the GitHub repository's project board and issue tracker. The most consequential schedule event was the resolution pivot in mid-April: initial training at the dataset's native 64×64 resolution produced a frozen-ResNet test accuracy substantially below expectations, prompting a four-day retraining of all three models at 224×224 resolution before downstream experiments could proceed.

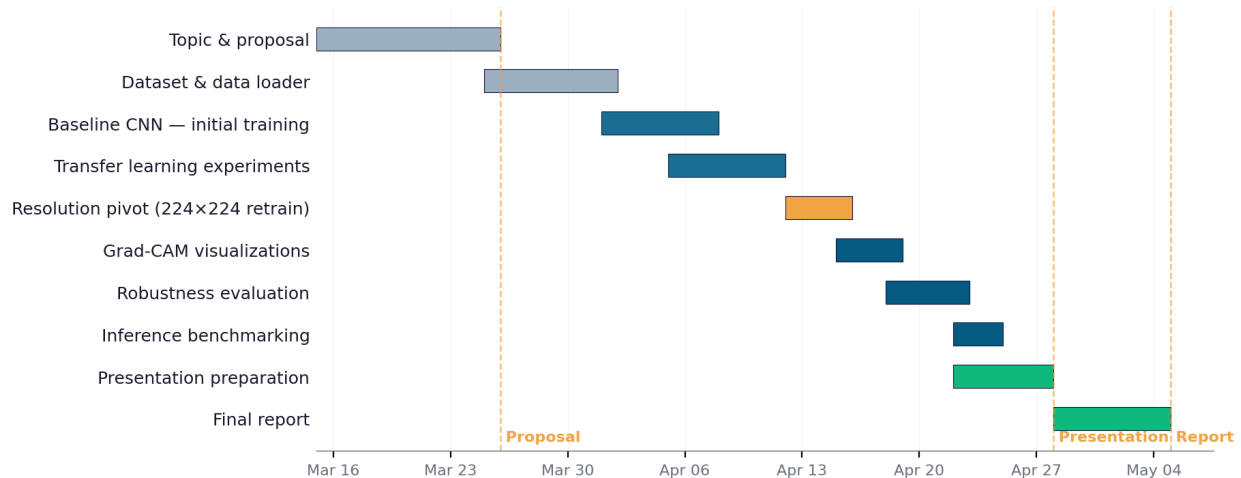


Figure 1. Project schedule. Dashed vertical lines mark hard deadlines.

3. Dataset

EuroSAT [1] is a publicly available labeled satellite image dataset derived from Sentinel-2. It contains 27,000 RGB images approximately evenly distributed across 10 land-cover classes (Figure 2). Each image is 64×64 pixels at the native resolution, and Sentinel-2's ground sampling distance is 10 m per pixel — so each image covers roughly 640 × 640 m of terrain. The dataset was split 70 / 15 / 15 (train/val/test), stratified by class, with a fixed seed of 42. Two transform pipelines operate over the same indices: an augmented pipeline (random horizontal flip + ±15° rotation) for training, and a clean pipeline for validation, testing, and all robustness experiments. Both pipelines resize to 224 × 224 by bilinear interpolation (rationale in §5).



Figure 2. One example image from each of the ten EuroSAT classes (64×64 native).

4. Network Architectures and Training

Three convolutional networks were trained, in increasing order of reliance on pretrained knowledge (Figure 3). The custom CNN is a three-block from-scratch network: each block is Conv 3×3 + BatchNorm + ReLU + MaxPool, with channel progression $3 \rightarrow 32 \rightarrow 64 \rightarrow 128$. Adaptive average pooling collapses any spatial size to 1×1 before two FC layers ($256 \rightarrow 10$), giving 129,290 trainable parameters. The frozen ResNet18 [2] loads torchvision's ImageNet-pretrained weights, sets `requires_grad = False` on every backbone parameter, and replaces the 1000-class head with a fresh 10-class linear layer (5,130 trainable parameters). The fine-tuned variant uses the same architecture and ImageNet initialization but allows all 11.18 M parameters to update, with learning rate reduced to 1×10^{-4} to avoid disrupting the pretrained features [4].

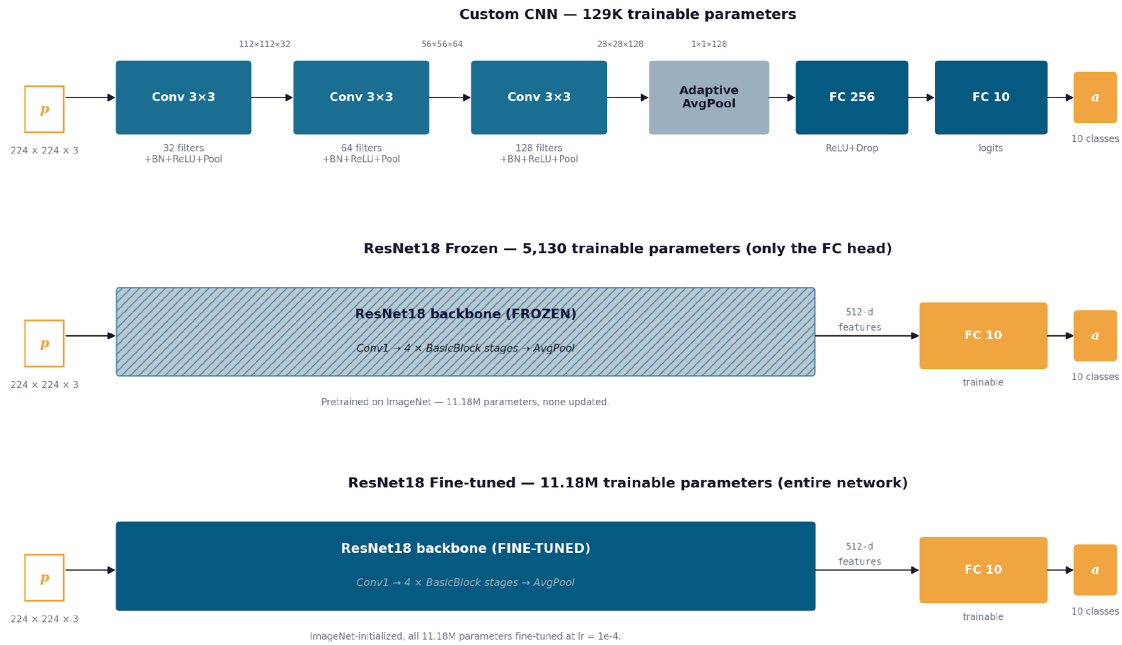


Figure 3. The three architectures, in increasing order of reliance on pretrained knowledge.

All three were trained by stochastic gradient descent with the Adam optimizer [3] using cross-entropy loss.

5. Experimental Setup

PyTorch 2.2 was used as the framework. Training ran on an NVIDIA GPU; inference benchmarks covered both that GPU and the host CPU. The data loader (`data_setup.py`, Listing 2) constructs two `ImageFolder` instances over the same directory — one with augmented transforms, one with clean — and applies the same stratified split to both, guaranteeing identical train/val/test image membership across pipelines. `DataLoader` uses 4 worker processes and `pin_memory = True`. Batch size was 64: smaller batches were noisy, larger batches exhausted GPU memory at 224×224 for ResNet. All three models trained for 25 epochs, with training loss plateauing by epoch 15 (baseline) or 20 (ResNet variants).

Initial training was performed at the dataset's native 64×64 resolution. The frozen ResNet reached only 81.98% test accuracy in this configuration — substantially below the baseline CNN's 92.59%. The hypothesized cause was architectural: ResNet18 has five spatial-reduction stages, so its deepest feature map has resolution $\text{input_size} / 32$. At 64×64 input that is only 2×2 , after which global average pooling discards almost all spatial information. All three models were retrained with input upsampled to 224×224 . Frozen ResNet improved by +9.21 points to 91.19%, this supports the hypothesis that the native 64×64 resolution was too small for ResNet18's downsampling structure. The fine-tuned variant gained +1.06 to 98.47%; the baseline lost 1.97 points to 90.62% (its adaptive pooling makes it less sensitive to input resolution). All subsequent experiments use 224×224 input.

Top-1 accuracy on the held-out test split is the primary metric. Per-class precision, recall, and F1 are computed via `scikit-learn`. Robustness is measured by re-evaluating each trained model against test images perturbed by one of five degradations at progressively larger severities (no retraining). Inference speed is measured by mean latency over 200 timed forward passes (after 20 warmup passes) at batch size 1, with explicit CUDA synchronization around each timed pass.

6. Results

6.1 Clean test accuracy

Table 1 reports clean-test results at 224×224 . The custom CNN at 90.62% essentially ties the frozen ResNet at 91.19%, with the fine-tuned ResNet leading by 7.28 percentage points at 98.47%. The 129K-parameter custom CNN matches the frozen ResNet despite the latter drawing on 11.18 M pretrained parameters. The gap between frozen and fine-tuned is substantial, indicating that ImageNet features transfer reasonably to satellite imagery but require domain-specific adaptation to reach the dataset's practical accuracy ceiling.

Model	Trainable parameters	Test accuracy
Custom CNN	129,290	90.62%
ResNet18 Frozen	5,130	91.19%
ResNet18 Fine-tuned	11,181,642	98.47%

Table 1. Clean test accuracy and trainable parameter count for the three models at 224×224 input.

6.2 Confusion analysis

Figure 4 places the three confusion matrices side by side. The baseline's weakest class is Highway at 72% recall, with confusions spread across River and Industrial — all three classes share elongated linear features in overhead view. The frozen ResNet's most striking failure is that 12% of River examples are misclassified as Highway: ImageNet's pretrained features were never exposed to overhead views of water versus asphalt, where both appear as thin dark strips. Fine-tuning reduces this confusion from 12% to 2%, the cleanest example of domain adaptation in the project.



Figure 4. Confusion matrices, left to right: custom CNN (90.62%), frozen ResNet (91.19%), fine-tuned ResNet (98.47%). Note the 12% River → Highway confusion (row 9) for the frozen model, dropping to 2% after fine-tuning.

6.3 Grad-CAM interpretability

Grad-CAM [5] heatmaps (Figure 5) reveal where each model attends. Three observations: First, feature-map resolution drives heatmap quality, the baseline CNN has 8×8 maps at 64×64 input (28×28 at 224×224), while ResNet has only 2×2 (7×7 at 224×224) due to five stride-2 stages. Second, channel depth and feature semantics differ: the baseline's 128 last-layer channels learned from EuroSAT alone encode low-level features, producing visually sharp heatmaps; ResNet's 512 deep semantic channels encode high-level patterns from ImageNet and produce diffuse, distributed attention. Third, the most accurate model (fine-tuned ResNet) produces the least visually interpretable heatmaps. In this experiment, visually sharper Grad-CAM maps did not correspond to higher test accuracy.

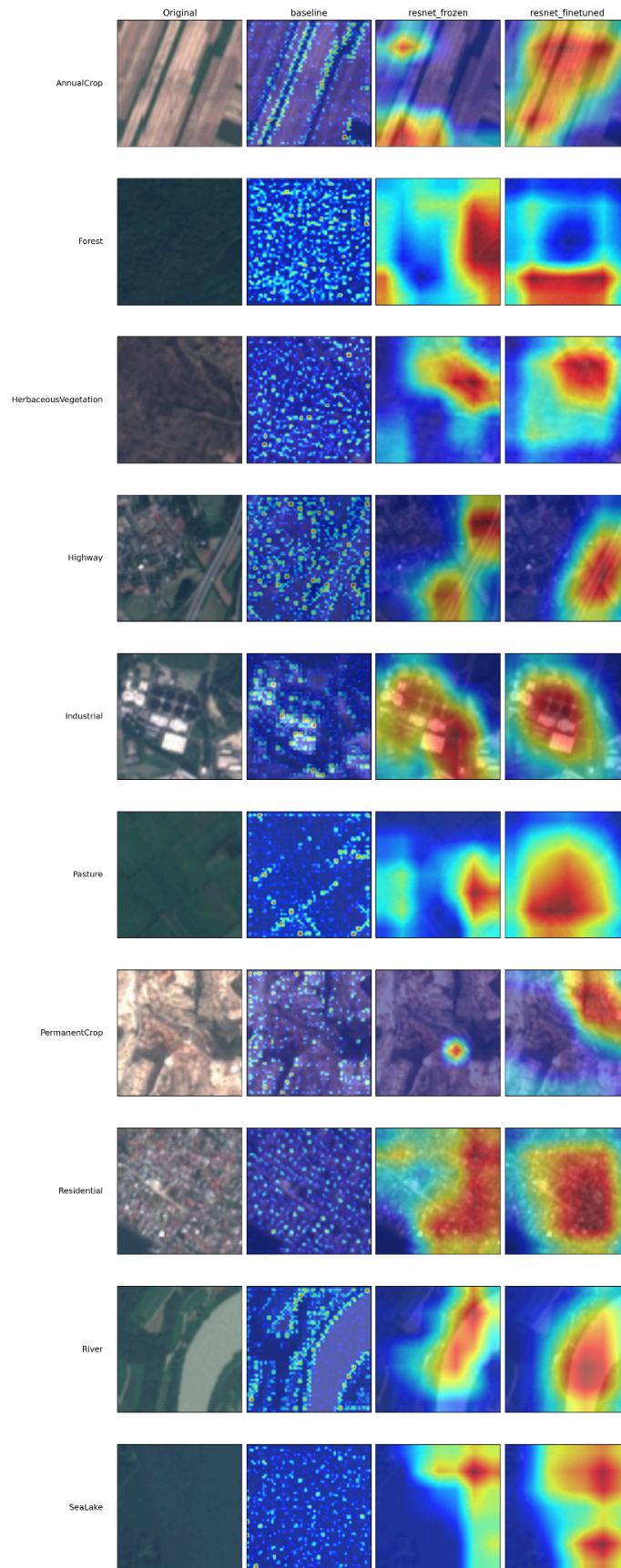


Figure 5. Grad-CAM heatmaps at 224×224 . Columns: original image, baseline CNN, frozen ResNet, fine-tuned ResNet. Rows: ten EuroSAT classes.

6.4 Robustness

Each model was evaluated against five degradations: Gaussian noise, Gaussian blur, resolution loss (downsample then upsample), rotation, and brightness scaling. All applied at test time with no retraining. Figure 6 shows accuracy curves for the four most informative degradations.

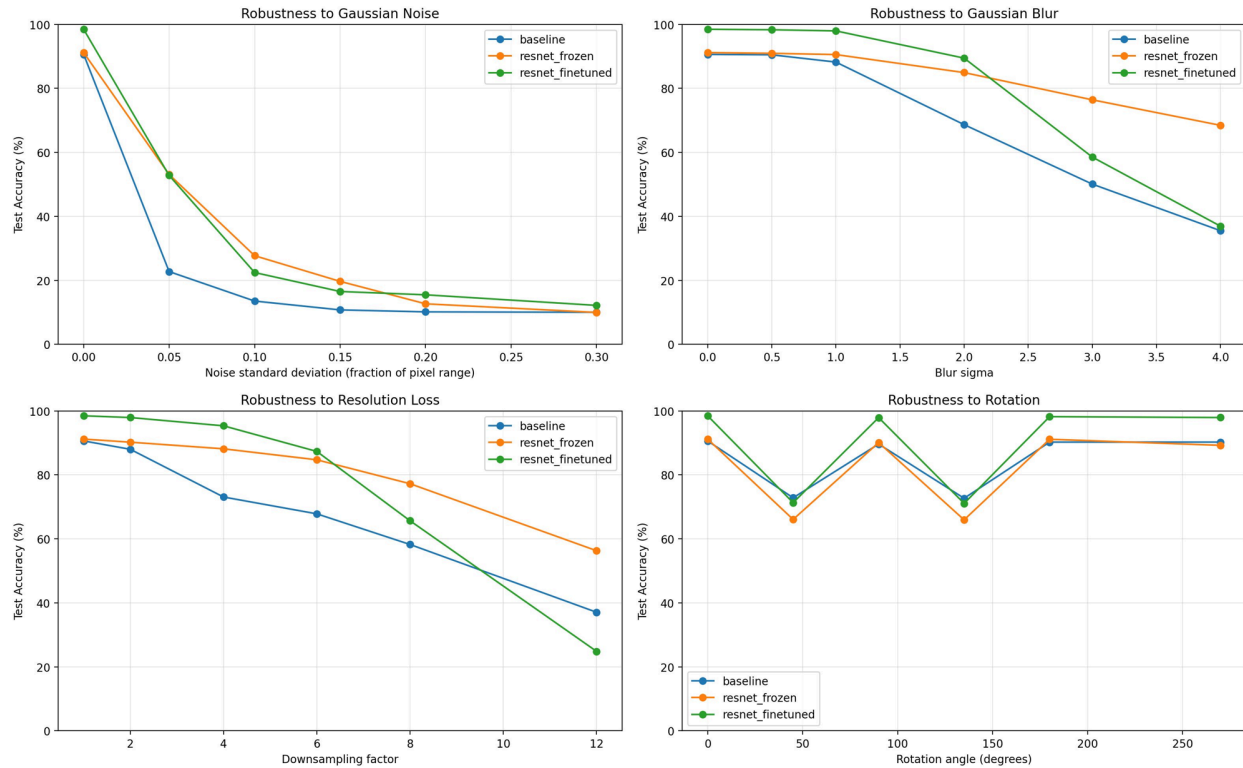


Figure 6. Robustness curves. Top row: Gaussian noise (left), Gaussian blur (right). Bottom row: resolution loss (left), rotation (right).

The blur and resolution-loss curves reveal a crossover effect: the fine-tuned ResNet is dominant at low severity but is overtaken by the frozen ResNet at extreme degradation. Frozen ImageNet features were trained on a more diverse corpus and generalize more robustly to out-of-distribution inputs than fine-tuned features specialized to the EuroSAT distribution. This is a clear accuracy-versus-robustness tradeoff.

The rotation curves expose a separate methodological lesson. All three models perform well at multiples of 90° and lose roughly 25 points at 45° / 135°. The cause is that training augmentation included only $\pm 15^\circ$ rotation plus horizontal flips, so the models effectively never saw off-axis imagery. They learned that satellite features are grid-aligned, not that they are rotation-invariant. This robustness ceiling is set entirely by training augmentation.

The Gaussian noise curve is even more striking: at $\sigma = 0.05$, a perturbation barely visible to a human accuracy drops from above 90% to 22% (baseline) or 53% (ResNet variants). By $\sigma = 0.15$, all three models are below 20% (chance for a 10-class problem). No noise augmentation was applied during training, and the models never learned invariance to it.

6.5 Inference benchmarks

Table 2 reports inference latency and throughput at batch size 1, with 200 timed passes after 20 warmup passes and explicit CUDA synchronization around each timed pass. The custom CNN runs $\sim 3.8\times$ faster than either ResNet on GPU and $\sim 1.5\times$ faster on CPU; both architectures become memory-bandwidth bound on CPU, narrowing the gap. Frozen and fine-tuned ResNets are identical at inference (same architecture, same parameter count). For edge deployment, the custom CNN's $\sim 87\times$ smaller parameter count and $4\times$ faster GPU inference present a meaningful tradeoff against its eight-percentage-point accuracy deficit.

Model	Device	Latency (ms)	Throughput	Total params
Custom CNN	GPU	0.51 ± 0.02	1,959 img/s	129K
Custom CNN	CPU	21.5 ± 3.1	47 img/s	129K
ResNet18 Frozen	GPU	1.93 ± 0.05	519 img/s	11.2M
ResNet18 Frozen	CPU	31.3 ± 1.4	32 img/s	11.2M
ResNet18 Fine-tuned	GPU	1.92 ± 0.04	522 img/s	11.2M
ResNet18 Fine-tuned	CPU	31.1 ± 0.8	32 img/s	11.2M

Table 2. Inference latency (mean $\pm$ std) and throughput at batch size 1.

7. Summary and Conclusions

Four conclusions emerge from the results.

- **Fine-tuning wins decisively on clean data.** 98.47% test accuracy, +7.28 over frozen transfer learning. Domain adaptation matters substantially for satellite imagery.
- **The custom CNN is surprisingly competitive.** 90.62% accuracy with 129K trainable parameters and $\sim 4\times$ lower GPU latency offers a meaningful tradeoff for compute-constrained deployment.
- **Clean accuracy does not predict robustness.** The fine-tuned ResNet, the most accurate on clean data, becomes the most fragile under severe blur and resolution loss. Specialized features generalize worse than generic ImageNet features at the distribution edge.
- **Augmentation defines the robustness ceiling.** The rotation zigzag and noise-induced collapse trace directly to what was absent from the training augmentation pipeline. Robustness must be designed in from the start.

Natural future work: expand training augmentation to full 0–360° rotation plus noise, blur, and brightness jitter; evaluate cross-geographic generalization (train on EuroSAT, test on a U.S. or Asian dataset); benchmark on edge hardware (Jetson Nano, Raspberry Pi); evaluate on actual lower-altitude drone imagery; and explore rotation-equivariant architectures [6] to address the rotation zigzag structurally rather than purely through augmentation.

8. References

- [1] Helber, P., Bischke, B., Dengel, A., and Borth, D. (2019). "EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification." *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(7), 2217–2226.
- [2] He, K., Zhang, X., Ren, S., and Sun, J. (2016). "Deep Residual Learning for Image Recognition." *CVPR*, 770–778.
- [3] Kingma, D. P., and Ba, J. (2015). "Adam: A Method for Stochastic Optimization." *ICLR*.
- [4] Yosinski, J., Clune, J., Bengio, Y., and Lipson, H. (2014). "How Transferable are Features in Deep Neural Networks?" *NeurIPS*, 27.
- [5] Selvaraju, R. R., et al. (2017). "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization." *ICCV*, 618–626.
- [6] Cohen, T. S., and Welling, M. (2016). "Group Equivariant Convolutional Networks." *ICML*, 2990–2999.
- [7] Hagan, M. T., et al. ECEN 5743 — Deep Learning course materials (Spring 2026).
- [8] Gildenblat, J. (2021). pytorch-grad-cam GitHub: <https://github.com/jacobgil/pytorch-grad-cam>
- [9] Project repository: <https://github.com/Earthburglar/Final-Project-Jacob-Garrison>

Appendix A. Code Listings

The complete code is available in the project GitHub repository [9]. The repository contains seven Python modules totaling approximately 1,800 lines: `data_setup.py` (data loading and splits), `models.py` (the three architectures), `train.py` (training loop), `evaluate.py` (test metrics and confusion matrices), `visualize.py` (Grad-CAM), `robustness.py` (test-time perturbations), and `benchmark.py` (inference timing). Listings 1 and 2 below present the two most distinctive components — the model factory and the dual-pipeline data loader — and Listing 3 shows the inference-benchmarking methodology, where care must be taken to measure kernel execution rather than asynchronous queue submission.

A.1 Repository structure

```
Final-Project-Jacob-Garrison/
├── Code/
│   ├── data_setup.py    # ImageFolder loading, transforms, stratified split
│   ├── models.py        # BaselineCNN + ResNet18 builders + get_model() factory
│   ├── train.py         # Unified training loop, --model dispatch
│   ├── evaluate.py       # Test-set metrics and confusion matrices
│   ├── visualize.py      # Grad-CAM hooks and figure generation
│   ├── robustness.py     # Five degradations × seven severity levels
│   └── benchmark.py      # CPU/GPU latency and throughput timing
├── Proposal/, Final-Presentation/, Final-Project-Report/
└── README.md            # Setup and execution instructions
```

A.2 Listing 1 — Model factory (`models.py`)

The `BaselineCNN` class and the `get_model()` dispatch.

```
class BaselineCNN(nn.Module):
    """Three-block CNN with adaptive pooling.

    Adaptive pooling lets the same architecture accept inputs of either
    64x64 or 224x224 without modifying the FC layer dimensions.
    """
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            self._block(3, 32),
            self._block(32, 64),
            self._block(64, 128),
        )
        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128, 256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.3),
            nn.Linear(256, num_classes),
        )

    @staticmethod
    def _block(in_c, out_c):
        return nn.Sequential(
            nn.Conv2d(in_c, out_c, 3, padding=1),
            nn.BatchNorm2d(out_c),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
        )

    def forward(self, x):
        return self.classifier(self.pool(self.features(x)))

def build_resnet(freeze_backbone: bool, num_classes: int = 10):
    """Load ImageNet-pretrained ResNet18 and replace the classifier head.
```



```

If freeze_backbone, every parameter except the new FC has requires_grad=False.
"""
model = torchvision.models.resnet18(
    weights=torchvision.models.ResNet18_Weights.DEFAULT)
if freeze_backbone:
    for p in model.parameters():
        p.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, num_classes)
return model

```

```

def get_model(name: str) -> nn.Module:
    """Factory dispatch from CLI name to a fresh module."""
    if name == "baseline": return BaselineCNN(num_classes=10)
    if name == "resnet_frozen": return build_resnet(freeze_backbone=True)
    if name == "resnet_finetuned": return build_resnet(freeze_backbone=False)
    raise ValueError(f"Unknown model: {name}")

```

A.3 Listing 2 — Dual-transform data loader (data_setup.py)

Two ImageFolder copies share the same data and the same indices but apply different transforms.

```

def get_dataloaders(data_dir, batch_size=64, image_size=224, seed=42):
    """Returns (train_loader, val_loader, test_loader, class_names)."""
    train_tfm = transforms.Compose([
        transforms.Resize((image_size, image_size)),
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomRotation(15),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])
    eval_tfm = transforms.Compose([
        transforms.Resize((image_size, image_size)),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])

    train_ds_full = ImageFolder(data_dir, transform=train_tfm)
    eval_ds_full = ImageFolder(data_dir, transform=eval_tfm)

    targets = np.array(train_ds_full.targets)
    rng = np.random.default_rng(seed)
    train_idx, val_idx, test_idx = stratified_split(
        targets, rng, train=0.70, val=0.15)

    return (
        DataLoader(Subset(train_ds_full, train_idx), batch_size=batch_size,
            shuffle=True, num_workers=4, pin_memory=True),
        DataLoader(Subset(eval_ds_full, val_idx), batch_size=batch_size,
            shuffle=False, num_workers=4, pin_memory=True),
        DataLoader(Subset(eval_ds_full, test_idx), batch_size=batch_size,
            shuffle=False, num_workers=4, pin_memory=True),
        train_ds_full.classes,
    )

```

A.4 Listing 3 — Inference benchmark (benchmark.py)

Warmup passes and CUDA synchronization are essential for accurate GPU latency measurement.

```

def benchmark_model(model, device, warmup_passes=20, timed_passes=200, input_size=224):
    """Returns (mean_ms_per_image, std_ms, throughput_imgs_per_sec)."""
    model.eval()
    is_cuda = device.type == "cuda"
    dummy = torch.randn(1, 3, input_size, input_size, device=device)

    # Warmup: trigger kernel JIT, allocator fast-path, BatchNorm steady state
    with torch.no_grad():
        for _ in range(warmup_passes):
            _ = model(dummy)
    if is_cuda: torch.cuda.synchronize()

```

```

# Timed passes – synchronize before AND after each
times_ms = []
with torch.no_grad():
    for _ in range(timed_passes):
        if is_cuda: torch.cuda.synchronize()
        start = time.perf_counter()
        _ = model(dummy)
        if is_cuda: torch.cuda.synchronize()
        times_ms.append((time.perf_counter() - start) * 1000.0)

mean_ms = float(np.mean(times_ms))
return mean_ms, float(np.std(times_ms)), 1000.0 / mean_ms

```

All experiments can be reproduced from a clean checkout: download EuroSAT_RGB to data/EuroSAT_RGB and run Code/run_all.sh. Total wall-clock is approximately two hours on a workstation with one mid-range GPU, dominated by the fine-tuned ResNet training. Per-epoch loss and accuracy are written to JSON in the results/ directory for every run.